> Is there a bash tool that parses .sh files looking for functions and aliases?

There's no single standard tool named specifically for this, but several approaches work well:

Dedicated tools

- `shfmt` (mvdan/sh) — not primarily a parser for listing functions, but it can parse and format shell scripts reliably, including bash.
- `bash-language-server` — LSP server; not a CLI lister, but it parses functions/aliases for editor features.
- `shellcheck` — `shellcheck -f json` (or `-f gcc`) parses the file; it doesn't emit a function/alias list directly, but you can combine with its AST via `shellcheck --format=json1` for some structure.
- `bashate` — style checker, not a symbol lister.

Simple, reliable one-liners

For **functions**:

```bash
grep -nE '^\s*(function\s+)?[a-zA-Z_][a-zA-Z0-9_]*\s*\(\)' file.sh
```

For **aliases**:

```bash
grep -nE "^\s*alias\s+[a-zA-Z_][a-zA-Z0-9_-]*=" file.sh
```

More robust: use bash's own parser

You can source in a subshell and introspect:

```bash
```

```
bash -c 'source ./file.sh >/dev/null 2>&1; declare -F; alias'
```

Caveats: this *executes* top-level code, so only do it on trusted files. `declare -F` lists function names; `alias` lists aliases (aliases only work in interactive-ish contexts, so you may need `shopt -s expand_aliases`).

Programmatic parsing

- `bashlex` (Python) — a real bash parser (Python library). You can walk the AST to extract function definitions and alias assignments accurately.
- `tree-sitter-bash` — grammar + bindings (Python, JS, Rust, etc.), great for building a symbol extractor that handles nested/edge cases.

Recommendation

- Quick check → `grep` patterns above.
- Correctness matters (comments, strings, nested) → `tree-sitter-bash` or `bashlex`.
- Just want to know what's defined after loading → `bash -c 'source file; declare -F'` (trusted files only).

Is '?' any special character in bash?

Yes, `?` is special in bash, but it has **two different meanings** depending on context:

1. Globbing (filename expansion) — the common one

In an unquoted word, `?` is a **wildcard matching exactly one character**:

```
bash

ls file?.txt      # matches file1.txt, filea.txt, but not file10.txt
rm temp?          # deletes temp1, tempA, ...
```

This applies during pathname expansion (like `*`, `[...]`).

2. In parameter expansion — conditional substring/operator

Inside `${...}`, `?` is used in the `${var?word}` form, which errors out if the variable is

unset:

```bash
echo "${HOME?must set HOME}"   # fine
echo "${UNSET?must set UNSET}" # bash: UNSET: must set UNSET
```

Variants: `${var?word}` (error if unset), `${var:?word}` (error if unset **or null**).

3. In glob/extglob matching within `[[ ]]` / `case`

It still means "one character" in patterns:

```bash
[[ abc == a?c ]]        # true
case $x in ??) echo two-chars;; esac
```

With `shopt -s extglob`, `?(pattern-list)` matches **zero or one** occurrence of the pattern.

What it is *not*

- Not a quoting character, not a redirection operator, not a control operator.
- Unlike `*`, `?` does **not** match a literal `.` at the start of a filename (dotfiles need explicit leading dot), same as `*`.

Quoting

To use a literal `?`, quote or escape it:

```bash
echo 'what?'      # literal
echo what\?       # literal
touch "a?b"       # literal filename
```

Summary

Context	Meaning

Context	Meaning
Unquoted word	single-character wildcard
`${var?word}` / `${var:?word}`	error if unset / unset-or-null
`[[ ]]` , `case` , extglob `?(...)`	pattern: one char / zero-or-one
Quoted	literal `?`

So yes — special, primarily as a **one-character glob**, secondarily as a **parameter-expansion operator**.

So it can give me problems with shopt iin certain scenarios

Yes — `?` can cause trouble specifically around `shopt` settings, mainly because `shopt` toggles behaviors that change how `?` is interpreted. Here are the real scenarios:

1. `extglob` — changes what `?(...)` means

This is the big one. With `shopt -s extglob` :

```bash
shopt -s extglob
ls ?(foo)bar      # matches "bar" or "foobar" (zero-or-one)
```

Without `extglob` , `?(foo)bar` is parsed as `?` (one char) followed by literal `(foo)bar` — completely different, and often an error or a failed match. So:

- Code written assuming extglob (`?(...)`) **breaks** if extglob is off.
- Code with a literal `?` before `(` behaves differently once someone enables extglob.
- **Gotcha:** extglob parsing is decided when a function/script is *read*, not when executed. If you define a function before `shopt -s extglob` , the `?(...)` inside it won't be treated as extglob even after you enable it. You must enable extglob *before* the function definition is parsed.

2. `nullglob` / `failglob` — unmatched `?` patterns

If a `?` glob matches nothing:

```bash
shopt -s nullglob   # "file?.txt" with no match → expands to nothing
shopt -s failglob   # → error: no match
# default           # → stays literal "file?.txt"
```

So the same `rm file?.txt` behaves three ways depending on `shopt`. With `failglob` a command can abort; with `nullglob` an argument can vanish (classic bug: `cmd file?.txt` losing its argument).

3. `globasciiranges`, `nocaseglob`, `dotglob` — affect matching, not `?` itself

These change *which* files `?` matches (case sensitivity, leading dots, range collation) but don't change `?`'s meaning.

4. `expand_aliases` — aliases can hide `?`

In non-interactive shells aliases are off by default. If someone `shopt -s expand_aliases` and has an alias, the alias expansion happens before globbing, so an alias could inject or alter `?` tokens.

5. `extdebug` / `globstar` — not `?`-specific

No direct effect on `?`.

Practical advice

- **Always quote literal `?`:** `"a?b"`, `'file?.txt'`, or `file\?.txt`. Don't rely on default glob behavior.
- **Set glob options explicitly** at the top of scripts (`shopt -s nullglob` etc.) rather than inheriting the user's interactive settings.
- **If you use extglob**, enable it *before* any function definitions that contain `?(...)` — it's a parse-time decision.
- **Don't toggle `shopt` inside functions expecting it to re-parse** earlier-defined code; it won't.

So the "problem with `shopt`" is mostly: `?`'s meaning depends on `extglob`, and whether an unmatched `?` pattern errors/vanishes/stays literal depends on `failglob`/`nullglob` — all `shopt`-controlled.

Shortest answer, how can I store a shopt and then restore it

```bash
saved=$(shopt -p extglob); shopt -s extglob; ...; eval "$saved"
```

`shopt -p <opt>` prints a ready-to-run `shopt -s/-u` command; `eval` restores it.

can I set more than one trap RETURN? Or will it be overwritten

Overwritten. A signal/`RETURN`/`EXIT` trap is a **single slot** — setting it again replaces the previous one. Only one `RETURN` trap can be active at a time.

To chain, capture and call the old one yourself:

```bash
prev=$(trap -p RETURN)          # save existing
mytrap() { ...; eval "$prev"; } # call yours, then the old
trap mytrap RETURN
```

Or just build one trap that calls multiple functions:

```bash
trap 'f1; f2' RETURN
```

Note: `trap -p RETURN` inside a function shows the trap for that context; `RETURN` traps fire when a function/sourced script returns, so be careful with nesting and `functrace`/`errtrace` shopt settings.

Shortest answer, is shopt inherited between subshells?

No — `shopt` settings are **not inherited** by subshells launched as separate `bash` processes (e.g. `bash -c`, scripts). They **are** inherited by `( ... )` subshells and `$(...)` command substitutions, since those fork the current shell.

One nuance: `inherit_errexit` affects `errexit` inheritance into `$(...)`; and `extglob`/other options set in the parent are visible in `( )` subshells but a *new* `bash` process only gets them via `BASHOPTS` if you export/inherit appropriately. Short version: same-shell subshells → yes; new `bash` process → no.

> Okay how can I get the current globstart shopt state without a subshell?

`globstar` — and the shortest no-subshell check:

```bash
shopt -q globstar && echo on || echo off
```

`shopt -q` sets exit status only (no output, no subshell). To get a boolean variable:

```bash
shopt -q globstar; gs=$?      # gs=0 → on, 1 → off
```

If you actually meant the literal `globstar` option name (typo above), same answer. If you want to *read it as text* without a subshell, note `shopt -p globstar` writes to stdout — capture it with a here-string/read instead:

```bash
read -r _ gs_state < <(shopt -p globstar)   # process substitution, not a command-subshell
```

But `shopt -q` + `$?` is the idiomatic no-subshell way.

> Shortest answer, can I eval a shopt -q and get the number in a variable?

```bash
shopt -q globstar; v=$?
```

No `eval` needed — `shopt -q`'s exit status is already the number, grabbed by `$?`.

> Can I?·

```
cleanup() {
  unset -f usage_msg allow_decorators
  if [ -z "NULLGLOB_STATE" ]; then
    shopt -s nullglob
  else
    shopt -u nullglob
  fi
```

Several bugs. Short list:

1. `-z "NULLGLOB_STATE"` is wrong

Missing `$` . `-z "NULLGLOB_STATE"` tests the **literal string** `"NULLGLOB_STATE"` , which is never empty → always false → always runs the `else` branch. You want:

```bash
if [ -z "$NULLGLOB_STATE" ]; then
```

But see #3 — even `"$NULLGLOB_STATE"` won't do what you think.

2. You never *set* the state variables

You reference `NULLGLOB_STATE` / `GLOBSTAR_STATE` but there's no code capturing the current state into them. You need, before changing anything:

```bash
shopt -q nullglob;  NULLGLOB_STATE=$?
shopt -q globstar;  GLOBSTAR_STATE=$?
```

3. The logic is inverted / the variables hold the wrong thing

If `NULLGLOB_STATE=$?` (0=on, 1=off), then `-z "$NULLGLOB_STATE"` is **always false** (it's `"0"` or `"1"` , never empty). So your `if/else` always takes the `else` . Correct test:

```bash
if [ "$NULLGLOB_STATE" -eq 0 ]; then shopt -s nullglob; else shopt -u nullglob; fi
```

Or simpler — don't store `$?`, store the ready-to-run command like earlier:

```bash
NULLGLOB_STATE=$(shopt -p nullglob)   # "shopt -u nullglob" or "shopt -s nullglob"
...
eval "$NULLGLOB_STATE"
```

(But that's a command-subshell — fine for capture, or use `read < <(...)` if you care.)

4. `unset -f usage_msg allow_decorators` — probably a typo

These look like functions you defined, but `unset -f` on a **non-existent** function is harmless; on the wrong names it won't do anything. Just make sure those names exist and aren't also variables. If they're variables, use `unset usage_msg allow_decorators` (no `-f`).

5. `unset -f NULLGLOB_STATE GLOBSTAR_STATE` — these are variables, not functions

`unset -f` targets functions. These are shell variables, so you want `unset NULLGLOB_STATE GLOBSTAR_STATE` (no `-f`). As written they stay set.

6. `trap 'cleanup' RETURN` fires on every function return

`RETURN` trap runs each time *any* function or sourced script returns, not just once. If `cleanup` runs on the return of some unrelated function, it restores shopt too early and unsets itself mid-run. You probably want:

- `trap 'cleanup' EXIT` (fires once at shell exit), **or**
- Keep `RETURN` but guard so it only runs at the point you intend, and re-arm as needed.

Also `RETURN` requires the trap to be set in the context that returns; inside a function it fires when *that* function (and nested ones, if `functrace` is on) returns.

Minimal corrected version

```bash
```

```
shopt -q nullglob;  NULLGLOB_STATE=$?
shopt -q globstar;  GLOBSTAR_STATE=$?

cleanup() {
  unset -f usage_msg allow_decorators
  if [ "$NULLGLOB_STATE" -eq 0 ]; then shopt -s nullglob; else shopt -u nullglob; fi
  if [ "$GLOBSTAR_STATE"  -eq 0 ]; then shopt -s globstar; else shopt -u globstar; fi
  unset NULLGLOB_STATE GLOBSTAR_STATE
  unset -f cleanup
}
trap 'cleanup' EXIT   # or RETURN if you truly want per-return
```

If you want no command-subshell capture, the `shopt -q; VAR=$?` form above is already subshell-free. Just remember to **capture before you mutate** and use `"$VAR" -eq 0`, not `-z`.